

ShopFlow

****E-Commerce Marketplace Platform****

Full-Stack B2C Solution - Spring Boot 3 + Next.js 16

Overview

ShopFlow is a production-ready B2C marketplace enabling sellers to publish products and customers to purchase them. Built with a modern tech stack featuring JWT authentication, real-time stock management, coupon system, order lifecycle, and seller dashboards.

****Author: yosrii****

****Repository: unknown****

****Stack: Java 21, Spring Boot 3.4.13, PostgreSQL/H2, Spring Security 6 + JWT, Springdoc OpenAPI, MapStruct, Next.js 16, TypeScript, Tailwind CSS 4****

Architecture

Technical Layers

Controller (REST) -> Service (@Transactional) -> Repository (JPA + Specifications) -> Entity

^

^

DTO + MapStruct (MapStruct)

GlobalExceptionHandler (@ControllerAdvice)

Project Structure

ShopFlow/

+-- shopflow.postman_collection.json

Data Flow - Order Creation

1. Client (Next.js) -> POST `/api/orders` with Bearer token
2. `JwtAuthenticationFilter` validates JWT + injects `SecurityContext`
3. `OrderController` @Valid -> `OrderService` @Transactional
4. Verify stock (Product + Variant), decrement atomically
5. Calculate totals, apply coupon, generate `ORD-YYYY-XXXX`
6. Create Order + OrderItems, clear Cart
7. `OrderRepository.save()` -> Hibernate flush -> PostgreSQL/H2
8. Return `OrderResponse` DTO 201

Frontend: `src/lib/api.ts` Axios client with auto-refresh on 401 -> POST `/api/auth/refresh` -> replay request.

Key Implementation Decisions

1. Multi-Level Stock Management (Product + Variants)

****Challenge**:** Spec requires variants (size/color) with individual stock + real-time cart/order validation.

****Solution**:** `Product.stock` (global) + `ProductVariant.stockSupplementaire` + `prixDelta`. Available stock = `product.stock + variant.stockSupplementaire`. Validated at cart add AND order creation (final check), atomic decrement in `@Transactional`. Returns 400 if insufficient. Hibernate LAZY + orphanRemoval prevents N+1.

2. Dynamic Filtering with Specifications + Pageable

****Challenge****: Full-text search across name/description/category/price + sorting + mandatory pagination.

****Solution****: `ProductSpecifications.java` with 6 combinable specifications: `hasCategory`, `hasSeller`, `priceBetween`, `isPromo`, `isActive`, `search(q)`. Dynamic query: `Specification.where(isActive(true)).and(hasCategory(...)).and(priceBetween(...))` + `productRepository.findAll(spec, pageable)`. Single method, testable, performant.

3. Order Number & Status Lifecycle

****Requirement****: `ORD-YYYY-XXXXX` + `PENDING -> PAID -> PROCESSING -> SHIPPED -> DELIVERED` (cancellation only `PENDING/PAID` with restock).

****Implementation****: `@PrePersist` generates unique order number. `OrderStatus` enum with controlled transitions in `OrderService.updateStatus()` and `cancel()` - validates state, restocks on cancellation. Business logic centralized in Service layer.

4. Stateless JWT Authentication (Access + Refresh)

****Stack****: Spring Security 6 + jjwt 0.12.6 (CVE-patched). HS384, 256-bit secret (env `JWT_SECRET` in prod).

****Tokens****: Access 1h + Refresh 7d. `JwtAuthenticationFilter` (`OncePerRequestFilter`) extracts Bearer, validates signature/expiry, injects `Authentication`. `SecurityConfig`: stateless, `permitAll` on public endpoints, `DaoAuthenticationProvider` + `BCrypt(12)`. Refresh endpoint re-issues access token. Frontend auto-handles 401 -> refresh -> retry.

5. Next.js 16 + TypeScript over Plain React

****Rationale****: App Router (file-based routing), SSR/SSG for SEO, image optimization, Server Components. React alone would require manual router + SSR setup. TypeScript mirrors Spring DTOs (Product, Cart, Order, Role) - prevents runtime type errors, enables autocomplete, reusable components (`OrderStatusBadge`, `RatingStars`, `ProductCard`).

6. Strict DTO Separation + MapStruct

Zero entity exposure. `ProductRequest` (`@Valid` constraints) <-> `Product` (JPA) <-> `ProductResponse` (enriched: seller name, categories, avg rating, discount %). `ProductMapper` `@Mapper(componentModel="spring")` handles conversions. Lombok `@Data/@Builder` reduces boilerplate. `GlobalExceptionHandler` centralizes 400/404.

7. Dual Database: PostgreSQL (Prod) / H2 (Dev)

`application.yml` with profiles: `dev` (default) -> H2 in-memory + `create-drop` + console. `prod` -> PostgreSQL + HikariCP + `update`. Switch via `mvn spring-boot:run -Dspring-boot.run.profiles=prod` with `DB_USERNAME`/`DB_PASSWORD` env vars.

Resolved Technical Challenges

API Endpoints (35 Total)

Postman collection: `shopflow.postman_collection.json` (20 requests with `{{base}}` + `{{accessToken}}` variables).

Frontend Routes (11 Pages)

Deployment

Development

```
```bash
```

```
mvn spring-boot:run
```

**H2 Console: <http://localhost:8080/h2-console> (jdbc:h2:mem:shopflow, sa,**

## Frontend (port 3000)

```
cd frontend
echo "NEXT_PUBLIC_API_URL=http://localhost:8080" > .env.local
npm install && npm run dev
```
```

Production

```
```bash
```

```
sudo systemctl start postgresql
```

## Backend

```
export DB_USERNAME=shopflow
export DB_PASSWORD=your_password
export JWT_SECRET=$(openssl rand -base64 32)
mvn spring-boot:run -Dspring-boot.run.profiles=prod
```

## Frontend

```
cd frontend
echo "NEXT_PUBLIC_API_URL=https://api.yourdomain.com" > .env.production
npm run build && npm start
```
```

Quality & Standards

- **Validation**: Bean Validation 3 (@Valid, @NotNull, @Email, @Size) on all DTOs
- **Documentation**: Swagger UI at `/swagger-ui.html` (Springdoc 2.8.13)
- **Error Handling**: `@ControllerAdvice` -> consistent JSON error responses
- **Code Style**: Lombok, MapStruct, consistent package structure
- **Security**: CORS configured for localhost:3000, stateless JWT, BCrypt(12)
- **Testing Ready**: `spring-boot-starter-test`, `spring-security-test`, Testcontainers compatible

Demo Accounts (Seeded via data.sql)

License

MIT - See LICENSE file.

****ShopFlow - yosrii - 2026 - 67 backend files + 11 frontend routes - BUILD SUCCESS****